

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

REPORT

Laboratory work No. 2

Meet the OS You Will Build

Rotaru Anastasia, FAF-242

Course: **Operating Systems**

Checked by: Reițman Patricia

Chişinău, 2026

Contents

1 Goal of the work **2**
1.1 Working environment 2

2 Part 1. Build it and boot it **2**

3 Part 2. Using xv6 as a Unix **3**
3.1 Observations 4

4 Part 3. Reading the source **4**
4.1 Observations 5

5 Part 4. My first xv6 program **6**
5.1 The source code 6
5.2 Adding it to the build 6
5.3 Build and test 6
5.4 Observations 7

6 Conclusion **7**

1 Goal of the work

The teaching operating system of this course is **xv6**, a small but real Unix (a modern re-implementation of Unix Version 6 in about 9 000 lines of C) that runs on the RISC-V architecture inside the QEMU emulator. In this lab I built xv6 from source, booted it, used it as a small Unix, read parts of its source code and wrote my first program that runs inside it. For every part I included the commands and the output I obtained and answered the observation questions in my own words.

1.1 Working environment

```
uname -a
which qemu-system-riscv64 riscv64-linux-gnu-gcc make git
```

```
Linux UbuntuLab 7.0.0-34-generic #34-Ubuntu SMP PREEMPT_DYNAMIC Wed Sep  2 14:34:54
    UTC 2026 aarch64 GNU/Linux
```

The lab was done on the Ubuntu virtual machine (hostname `UbuntuLab`, user `vboxuser`) running under VirtualBox. The kernel is Linux 7.0.0 for the `aarch64` (ARM64) architecture. This means that QEMU emulates a RISC-V computer on top of an ARM64 one, and the RISC-V cross-compiler (`riscv64-linux-gnu-gcc`) produces code for a processor different from the one of the VM.

On a fresh image the toolchain had to be installed first:

```
sudo apt update
sudo apt install -y git build-essential gdb-multiarch qemu-system-misc \
     gcc-riscv64-linux-gnu binutils-riscv64-linux-gnu
sudo apt install -y qemu-system-riscv
```

The second command was needed because on this Ubuntu release the package `qemu-system-misc` no longer contains the RISC-V emulator: it is in the separate package `qemu-system-riscv`. Without it the build finished, but `make qemu` failed with `qemu-system-riscv64: No such file or directory`. After the installation, `which qemu-system-riscv64` printed `/usr/bin/qemu-system-riscv64`.

2 Part 1. Build it and boot it

```
git clone https://github.com/mit-pdos/xv6-riscv ~/xv6-riscv
cd ~/xv6-riscv
ls
make qemu
```

```
LICENSE  Makefile  README  kernel  mkfs  test-xv6.py  user

qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -
    nographic ...

xv6 kernel is booting

hart 1 starting
```

```
hart 2 starting
init: starting sh
$
```

The command `make qemu` does three things: it compiles the kernel into `kernel/kernel`, compiles every user program and packs them together with the file `README` into a disk image `fs.img` (with the helper tool `mkfs`), and finally starts QEMU with that kernel and that disk. The messages `hart 1 starting` and `hart 2 starting` show that xv6 started the other two of the three emulated processor cores (`-smp 3`). Then the first process, `init`, started the shell and the prompt `$` appeared, so the OS booted successfully.

To leave xv6 I opened the QEMU monitor with Ctrl-A followed by C and typed `quit`. This is the QEMU exit sequence, not a command of xv6.

3 Part 2. Using xv6 as a Unix

At the xv6 prompt I ran the built-in programs.

```
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 36728
echo           2 4 35592
forktest       2 5 18136
grep           2 6 44184
init           2 7 35976
kill           2 8 35512
ln             2 9 35304
ls             2 10 42880
mkdir          2 11 35568
rm             2 12 35552
sh             2 13 58312
stressfs       2 14 36416
usertests      2 15 206800
grind          2 16 51824
wc             2 17 37728
zombie         2 18 34848
...
console        3 23 0
$ cat README
xv6 is a re-implementation of Dennis Ritchie's and Ken Thompson's Unix
Version 6 (v6).  xv6 loosely follows the structure and style of v6,
but is implemented for a modern RISC-V multiprocessor using ANSI C.
...
$ echo hello xv6
hello xv6
$ ls | grep c
cat            2 3 36728
echo           2 4 35592
wc             2 17 37728
sync           2 22 34944
console        3 23 0
$ wc README
48 336 2441 README
```

In the listing of `ls`, the three numbers after each name are the type of the entry (1 = directory, 2 = file, 3 = device), its inode number and its size in bytes. The last entry, `console`, is a device represented as a file. The command `wc README` counted 48 lines, 336 words and 2 441 bytes; the last number is the same as the size shown by `ls`. The pipe worked exactly as on Linux: only the lines of the output of `ls` that contain the letter `c` were kept by `grep`. I did not run the optional `usertests -q`.

3.1 Observations

1. Three programs that xv6 ships with.
From the listing: `ls` (lists a directory), `cat` (prints the content of a file) and `echo` (prints its arguments). Others are `grep`, `wc`, `mkdir`, `rm`, `kill` and the shell `sh`.

2. Which two OS features must exist for a pipe to work?
First, **processes**: in `ls | grep c` the shell has to create two separate processes (with `fork` and `exec`), one running `ls` and one running `grep`, and they run at the same time. Second, **communication between processes** (IPC): the kernel provides the system call `pipe()`, which creates a channel in kernel memory. The output of the first process is written into one end and the second process reads it from the other end, so the two processes can exchange data even though their memories are isolated from each other.

3. The xv6 shell compared with the Linux shell of Lab 1.
The xv6 shell follows the same basic model as the Linux one (prompt, commands, arguments, pipes, redirection), but it is much smaller and simpler: it has no history, no tab completion and no advanced scripting.

4 Part 3. Reading the source

After quitting QEMU I looked at the source tree. It has two main folders: `kernel/` is the operating system itself, which runs privileged (scheduler, memory, file system, system-call handlers), and `user/` holds the ordinary programs that run on top of it (`ls`, `cat`, `echo`, the shell).

```
sed -n "1,30p" user/cat.c
cat user/user.h
grep -n "sys_read\|sys_write" kernel/sysfile.c
```

```
#include "kernel/types.h"
#include "kernel/fcntl.h"
#include "user/user.h"

char buf[512];

void
cat(int fd)
{
  int n;

  while ((n = read(fd, buf, sizeof(buf))) > 0) {
    if (write(1, buf, n) != n) {
      fprintf(2, "cat: write error\n");
      exit(1);
```

```
    }
  }
  ...
```

```
// system calls
int fork(void);
int exit(int) __attribute__((noreturn));
int wait(int *);
int pipe(int *);
int write(int, const void *, int);
int read(int, void *, int);
int close(int);
int kill(int);
int exec(const char *, char **);
int open(const char *, int);
...
int getpid(void);
char *sys_sbrk(int, int);
int pause(int);
int uptime(void);
int sync(void);
```

```
69:sys_read(void)
83:sys_write(void)
```

The file `user/user.h` is the list of system calls that programs can use. In `cat.c`, the functions `read()` and `write()` look like ordinary C functions, but they are only declarations of requests to the kernel; the real work is done by the handlers `sys_read` and `sys_write` in `kernel/sysfile.c`.

4.1 Observations

1. Which system calls does `user/cat.c` use and what does each ask the kernel to do?
The program uses `open()`, which asks the kernel to open the file given on the command line and to return a file descriptor; `read()`, which asks the kernel to copy bytes from that descriptor into the buffer `buf`; `write()`, which asks the kernel to write the bytes to descriptor 1 (the screen); `close()`, which releases the descriptor; and `exit()`, which asks the kernel to terminate the process. When no argument is given, `cat` reads from descriptor 0 (the keyboard). The error messages are written with `fprintf(2, ...)`, which also ends in `write()` on descriptor 2.

2. Where is `sys_read` implemented?
In the file `kernel/sysfile.c`, at line 69 (`sys_write` is at line 83). The function takes its arguments from the user process (`argaddr`, `argint`, `argfd`) and calls `fileread()` to do the reading.

3. Difference between the code in `kernel/` and in `user/`.
The code in `kernel/` is the operating system: it runs in privileged mode and manages the hardware and the resources (processes, memory, files). The code in `user/` consists of ordinary programs that run without privileges and can reach the resources only by asking the kernel through system calls.

5 Part 4. My first xv6 program

I added a new user program, `sleep`, that pauses for a number of clock ticks by calling a system call that already exists in the kernel.

5.1 The source code

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1]));
  exit(0);
}
```

The program checks that exactly one argument was given; otherwise it prints the usage message on descriptor 2 (the error output) and exits with the code 1. If the argument is present, `atoi` converts the text to a number and the system call is invoked with it.

An important detail: the assignment calls the system call `sleep()`, but in the current version of xv6 that I cloned it was renamed to `pause()`. I found this in `user/user.h`, which contains `int pause(int);` and no `sleep`, so with the old name the program would not compile. The program itself is still called `sleep`.

5.2 Adding it to the build

In the `Makefile`, in the list `UPROGS`, I added a line for the new program right after `$U/_echo`. It must begin with a tab and end with a backslash:

```
sed -i 's|^\t\$U/_echo\\$|\t$U/_echo\\\n\t$U/_sleep\\|' Makefile
grep -n "_echo\|_sleep" Makefile
```

```
134:    $U/_echo\
135:    $U/_sleep\
```

5.3 Build and test

```
make qemu
```

```
... -c -o user/sleep.o user/sleep.c
riscv64-linux-gnu-ld ... -o user/_sleep user/sleep.o user/ulib.o user/usys.o ...
mkfs/mkfs fs.img README user/_cat user/_echo user/_sleep user/_forktest ...

xv6 kernel is booting

hart 2 starting
```

```
hart 1 starting
init: starting sh
$ sleep 10
$ sleep
usage: sleep <ticks>
```

The build compiled `user/sleep.c`, linked it as `user/_sleep` and included it in the disk image (`user/_sleep` appears in the `mkfs` command). In xv6, `sleep 10` paused for a short moment and then returned to the prompt, and `sleep` without an argument printed the usage message. Both behaviours are the expected ones.

5.4 Observations

The program is a good example of the split from Part 3. `sleep.c` lives in `user/` and contains no code that waits: it only calls `pause()`, which is a request to the kernel. The kernel, in `kernel/sysproc.c`, blocks the process for the requested number of timer ticks and wakes it up afterwards. The process is not running while it waits, so the CPU can be given to other processes, exactly as I saw with the sleeping process in the previous lab.

6 Conclusion

In this lab I built the teaching OS xv6 from source, booted it in QEMU and used its shell and its programs as a small Unix, including a pipe. Reading the source showed me how an OS is organised: `kernel/` is the privileged operating system, `user/` holds the ordinary programs, and the system calls declared in `user/user.h` are the only bridge between them. I found the implementation of `sys_read` in `kernel/sysfile.c` and finally I wrote and ran my own program, `sleep`, that calls a kernel service on an OS I had compiled myself.

Two practical lessons came out of the lab: the toolchain of a new Ubuntu release can differ from the one in the assignment (the RISC-V emulator is in a separate package), and so can the source of xv6 (the system call `sleep` is now called `pause`), so it is important to read the error messages and check the actual headers instead of copying commands blindly. Next week the changes will move to the kernel side as well.